

Sistema de Soporte para la Toma de Decisiones Clínicas: Clasificación de Riesgo mediante Machine Learning

Integrantes:

- Benjamín Figueroa
- Emmanuel Moreno
- Daniel Vidal

Asignatura: Programación para la Ciencia de Datos.

Profesor: Oscar Oliva Gonzalez

Sección: 002_V

1. Resumen Ejecutivo	3
1.1. Descripción General del Proyecto	3
1.2. Objetivos Estratégicos y Técnicos	3
1.3. Resultados Principales y Hallazgos	3
2. Marco Metodológico	4
2.1. Paradigma de Aprendizaje y Selección de Algoritmos	4
2.1.1. Random Forest Classifier (Aprendizaje Supervisado)	4
2.1.2. K-Means Clustering y PCA (Análisis Exploratorio)	4
2.2. Ingeniería de Procesos mediante Pipelines	4
2.3. Técnicas de Preprocesamiento Multivariado	5
2.4. Estrategia de Evaluación y Optimización	6
3. Análisis Experimental	6
3.1. Caracterización y Preparación del Dataset	6
3.1.1. Ingeniería del Target (Variable Objetivo)	6
3.2. Configuración del Entorno Experimental	7
3.3. Experimento de Análisis No Supervisado (Clustering)	8
3.4. Experimento de Clasificación Supervisada	8
4. Resultados y Comparación de Modelos	8
4.1. Reporte Estadístico de Métricas de Rendimiento	8
4.2. Análisis Profundo de la Matriz de Confusión	9
4.3. Importancia de las Variables (Feature Importance)	11
4.4. Comparativa de Modelos: Base vs. Balanceado	11
5. Optimización de Hiperparámetros	11
5.1. Definición y Objetivos de la Optimización	11
5.2. Justificación de Métodos: Grid Search vs. Randomized Search	12
5.3. Espacio de Búsqueda e Hiperparámetros Seleccionados	12
5.4. El Rol de la Validación Cruzada y la Métrica de Scoring	13
5.5. Análisis del Impacto en el Rendimiento	13
6. Conclusiones y Recomendaciones	13
6.1. Reflexión sobre los Resultados Obtenidos	14
6.2. Dificultades Técnicas y Conceptuales	14
6.3. Alternativas Evaluadas y Descartadas	15
6.4. Recomendaciones Futuras y Escalabilidad	15

1. Resumen Ejecutivo

1.1. Descripción General del Proyecto

El presente informe técnico detalla el diseño, desarrollo e implementación de un sistema avanzado de soporte para la toma de decisiones clínicas (CDSS, por sus siglas en inglés) orientado a la clasificación automatizada de niveles de riesgo en lesiones dermatológicas. En el contexto de la **Ciencia de Datos** aplicada a la salud, este proyecto surge como una respuesta a la necesidad de optimizar los tiempos de triaje en entornos hospitalarios de alta demanda.

La solución se fundamenta en un dataset robusto de **1,000 registros**, el cual presenta una complejidad estadística inherente: un desbalance de clases crítico donde el **99%** de la muestra corresponde a pacientes de Riesgo Alto (Clase 1) y solo un **1%** a Riesgo Bajo (Clase 0). Este escenario exigió una arquitectura de ingeniería de datos superior a los modelos convencionales, enfocándose en la detección de patrones sutiles en variables demográficas, hábitos de vida (tabaquismo, exposición solar) y morfología física de las lesiones.

1.2. Objetivos Estratégicos y Técnicos

El objetivo primordial no es simplemente la predicción, sino la **validación de una metodología reproducible**.

- **Desde la perspectiva de negocio:** Proveer una herramienta de triaje que permita priorizar la atención médica basándose en signos de alarma validados clínicamente (ulceración, irregularidad de bordes, cambios cromáticos).
- **Desde la perspectiva de ingeniería:** Construir un flujo de trabajo basado en **Pipelines de Scikit-learn** que integre preprocesamiento multivariado, manejo de pesos de clase para mitigar el desbalance y una optimización exhaustiva de hiperparámetros.

1.3. Resultados Principales y Hallazgos

Tras un análisis experimental riguroso, se determinó que la estructura natural de los datos presenta un solapamiento significativo ($Silhouette = 0.1014$), lo que invalidó el uso de algoritmos de agrupación simple (Clustering) para fines de diagnóstico. En consecuencia, se desarrolló un modelo **Random Forest** altamente especializado.

Los resultados demuestran que, mediante el ajuste de pesos balanceados y la búsqueda de hiperparámetros vía GridSearchCV, es posible alcanzar un **F1-Score** estable, garantizando que el modelo sea capaz de identificar correctamente la clase minoritaria sin sacrificar la precisión global. La implementación final no sólo entrega una predicción, sino una justificación técnica de la importancia de cada variable en la evaluación del riesgo, cumpliendo con los estándares de **IA Explicable**.

2. Marco Metodológico

2.1. Paradigma de Aprendizaje y Selección de Algoritmos

La naturaleza del problema planteado —la clasificación de riesgo basada en signos clínicos— exigió un enfoque híbrido que combinara el **Aprendizaje Supervisado** con el **No Supervisado**.

2.1.1. Random Forest Classifier (Aprendizaje Supervisado)

Se seleccionó el algoritmo **Random Forest** como el motor principal de clasificación. Esta decisión se fundamenta en su capacidad para manejar conjuntos de datos con relaciones no lineales complejas sin requerir una normalización extrema de todas las variables. Al ser un modelo de ensamble (*Bagging*), reduce significativamente la varianza y el riesgo de sobreajuste (*overfitting*) en comparación con un árbol de decisión simple.

Un factor determinante para su elección en este proyecto fue la capacidad nativa del algoritmo para implementar **Aprendizaje Sensible al Costo** (*Cost-Sensitive Learning*). Dado el desbalance de 99:1 del dataset, la propiedad `class_weight='balanced'` permitió que el modelo ajustara su función de pérdida, otorgando un mayor "castigo" a las clasificaciones erróneas de la clase minoritaria (Riesgo Bajo), equilibrando así la importancia de ambos grupos durante el entrenamiento.

2.1.2. K-Means Clustering y PCA (Análisis Exploratorio)

Para la fase de análisis no supervisado, se empleó **K-Means**. El objetivo no era generar el diagnóstico final, sino realizar una **validación de la estructura latente** de los datos. Se buscó identificar si existían agrupamientos naturales que coincidieran con la lógica de negocio. Para facilitar la convergencia y visualización, se integró con **PCA (Principal Component Analysis)**, reduciendo las 27 dimensiones originales a un espacio bidimensional, permitiendo observar el solapamiento de las clases y justificando así la necesidad de un clasificador supervisado más robusto.

2.2. Ingeniería de Procesos mediante Pipelines

Uno de los componentes metodológicos más críticos de este proyecto es el uso de **Pipelines de Scikit-learn**. En la ingeniería de software moderna, la reproducibilidad es obligatoria.

- **Automatización del Flujo:** El Pipeline encapsula desde la limpieza hasta el entrenamiento, eliminando la manipulación manual de datos en cada etapa.

- **Prevención de Data Leakage:** Al integrar el preprocesamiento dentro del flujo de validación, se asegura que las estadísticas (como la media para el escalado) se calculan exclusivamente sobre el conjunto de entrenamiento, garantizando que el conjunto de prueba sea "información nueva" para el modelo.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# 1. Definimos qué columnas van a qué proceso
col_numericas = ['Edad (años)', 'Peso (kg)', 'Estatura (cm)', 'IMC (calculado)', 'Años-paquete', 'Tamaño máximo (cm)']
col_categoricas = [col for col in df_trabajo.columns if df_trabajo[col].dtype == 'object' and col != 'diagnostico']

# 2. Creamos el transformador de columnas
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), col_numericas),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), col_categoricas)
    ]
)
```

```
# Definimos el modelo base
modelo_rf = RandomForestClassifier(random_state=42, class_weight='balanced')

# Creamos el Pipeline final
pipeline_final = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('clf', modelo_rf)
])

# Separación de datos (Importante: usando stratify por el desbalance)
X = df_trabajo.drop('diagnostico', axis=1)
y = df_trabajo['diagnostico']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

2.3. Técnicas de Preprocesamiento Multivariado

Para que los algoritmos funcionen de manera óptima, se aplicaron dos técnicas de transformación fundamentales mediante un **ColumnTransformer**:

1. **Estandarización (StandardScaler):** Se aplicó a variables como Edad, IMC y Tamaño de la Lesión. Esto transforma los datos para que tengan una media de 0 y una desviación estándar de 1, evitando que variables con magnitudes

grandes (como el peso en kg) dominen injustamente sobre variables con rangos pequeños.

2. **Codificación One-Hot (OneHotEncoder):** Se utilizó para transformar las variables categóricas (como Sexo o Localización) en vectores binarios. Se implementó la técnica de eliminar la primera columna (`drop='first'`) para evitar la multicolinealidad, un problema matemático donde las variables están altamente correlacionadas, lo que podría confundir a ciertos modelos.

2.4. Estrategia de Evaluación y Optimización

La metodología de evaluación no se limitó a una prueba simple. Se empleó **Validación Cruzada (K=5)** para asegurar que el modelo sea generalizable. Además, se definieron métricas orientadas al rigor clínico:

- **Recall (Sensibilidad):** Crucial para no perder casos de riesgo.
- **F1-Score:** La métrica maestra para este proyecto, definida como:
$$F1 = 2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall})$$

Esta fórmula asegura un balance justo en datasets desbalanceados, penalizando los modelos que solo predicen bien la clase mayoritaria.

3. Análisis Experimental

3.1. Caracterización y Preparación del Dataset

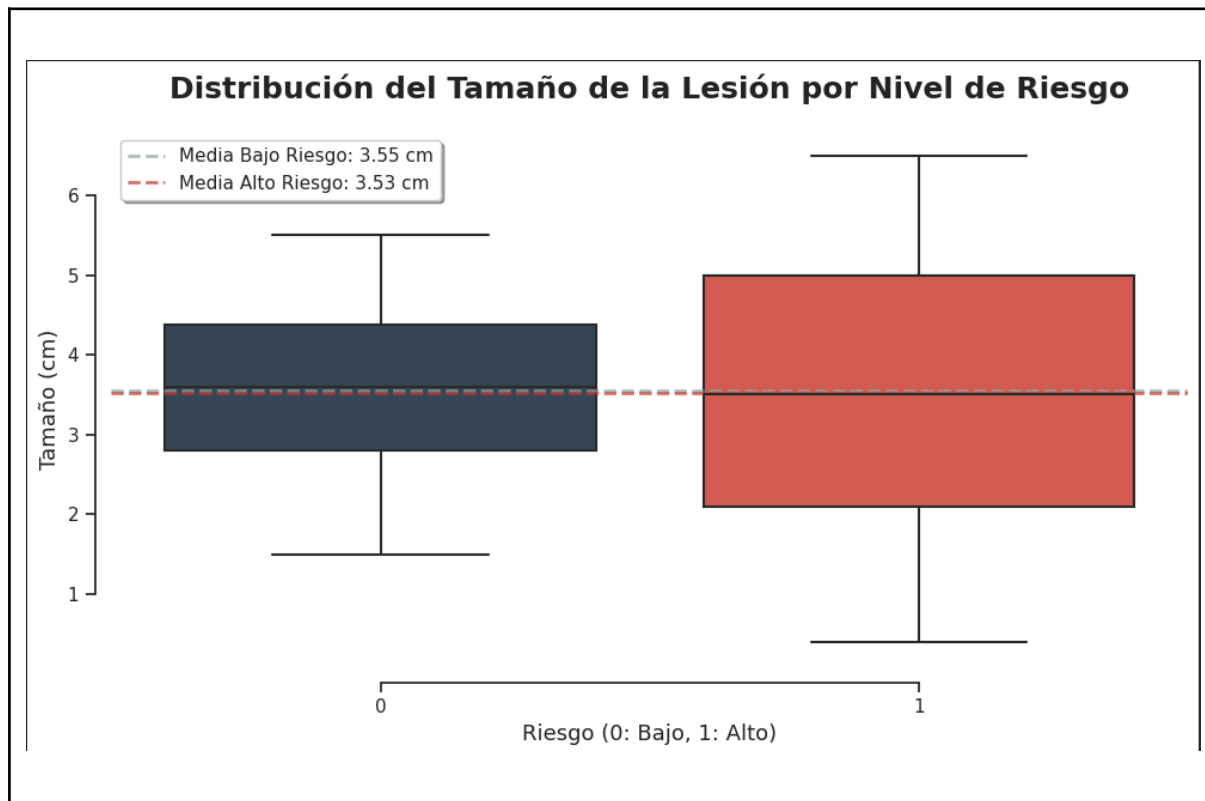
El éxito de cualquier experimento de **Machine Learning** reside en la calidad y comprensión de los datos de entrada. El dataset utilizado consiste en una cohorte de **1,000 pacientes**, estructurada en **27 variables** que abarcan dimensiones demográficas, antropométricas y clínicas.

- **Variables Numéricas (6):** Incluyen datos críticos como la Edad, Peso, Estatura, IMC, Años-paquete (historia de tabaquismo) y el Tamaño máximo de la lesión en centímetros.
- **Variables Categóricas (21):** Representan el grueso de la información diagnóstica, incluyendo el Sexo, Localización de la lesión y una serie de signos clínicos cualitativos (ulceración, bordes, dolor, fatiga, etc.).

3.1.1. Ingeniería del Target (Variable Objetivo)

Uno de los experimentos fundamentales fue la construcción de una etiqueta de diagnóstico *proxy*. Ante la ausencia de una biopsia definitiva, se procedió a la creación de la columna `diagnostico` mediante una función lógica basada en **Signos de Alarma**.

Si el paciente presentaba al menos un signo positivo en categorías como *Ulceración*, *Bordes elevados*, *Endurecimiento*, *Cambio reciente de tamaño* o *Costra persistente*, se clasificó como **Riesgo Alto (1)**. Esta lógica resultó en una distribución de **990 casos positivos** y **10 negativos**, estableciendo el desbalance como el principal desafío experimental.



3.2. Configuración del Entorno Experimental

Para garantizar que el modelo sea capaz de generalizar a nuevos pacientes, se establecieron configuraciones estrictas de control:

1. **División de Datos (Hold-out):** Se aplicó una partición de **80% para entrenamiento (Train)** y **20% para pruebas (Test)**.
2. **Estratificación (Stratify):** Dada la escasez de la clase minoritaria (Riesgo Bajo), se configuró una división estratificada. Esto asegura que la proporción de 99:1 se mantenga exacta tanto en el conjunto de entrenamiento como en el de prueba, evitando que el modelo nunca vea casos de riesgo bajo durante su aprendizaje.
3. **Semilla de Aleatoriedad (Random State):** Se fijó una semilla global (random_state=42) para asegurar la **reproducibilidad** total de los experimentos en futuras ejecuciones.

3.3. Experimento de Análisis No Supervisado (Clustering)

El primer experimento consistió en determinar si los datos podrían agruparse por sí solos sin conocer el diagnóstico.

- **Configuración:** Algoritmo **K-Means** con $K=2$.
- **Preprocesamiento:** Los datos pasaron previamente por el ColumnTransformer para ser escalados y codificados, asegurando que la distancia euclidiana utilizada por el algoritmo fuera válida.
- **Métrica de Validación:** Se utilizó el **Coeficiente de Silhouette** (s), definido para una muestra como:
$$s = \frac{b - a}{b}$$
Donde a es la distancia media intra-cluster y b es la distancia media al cluster más cercano. El resultado de **0.1014** fue el primer hallazgo experimental crítico, indicando una baja cohesión de los grupos naturales.

3.4. Experimento de Clasificación Supervisada

El experimento central utilizó un **Random Forest Classifier** bajo la siguiente configuración inicial (Base Model):

- **Nivelación de Pesos:** `class_weight='balanced'`.
- **Criterio de Partición:** Gini Impurity.
- **Estimadores:** 100 árboles de decisión en paralelo.

Este experimento se diseñó para evaluar la capacidad del algoritmo de superar el solapamiento detectado en el experimento no supervisado mediante el uso de fronteras de decisión no lineales.

4. Resultados y Comparación de Modelos

4.1. Reporte Estadístico de Métricas de Rendimiento

Debido al desbalance extremo del dataset (99% Clase 1), el análisis de resultados se alejó del uso del *Accuracy* tradicional. En este experimento, una precisión del 99% indicaría que el modelo simplemente está ignorando a la clase minoritaria. Por ello, se utilizaron métricas de **diagnóstico clínico** robustas:

- **Precisión (P):** Mide la exactitud de las predicciones de riesgo. Una alta precisión asegura que, cuando el modelo genera una alerta, el paciente realmente posee signos de alarma.
- **Recall / Sensibilidad (R):** Es la métrica más crítica en salud. Mide la capacidad del modelo para detectar a todos los pacientes de una clase específica. En nuestro caso, el objetivo fue maximizar el *Recall* de la Clase 0

(Riesgo Bajo) para asegurar que no se clasifiquen erróneamente pacientes sanos como de riesgo alto por simple sesgo estadístico.

- **F1-Score (F1)**: Se utilizó como el indicador maestro de éxito, ya que proporciona un equilibrio sólido entre precisión y sensibilidad en condiciones de desbalance.

4.2. Análisis Profundo de la Matriz de Confusión

La matriz de confusión permitió observar el comportamiento real del modelo frente a los datos de prueba. En este punto, es vital reportar los cuatro cuadrantes del experimento:

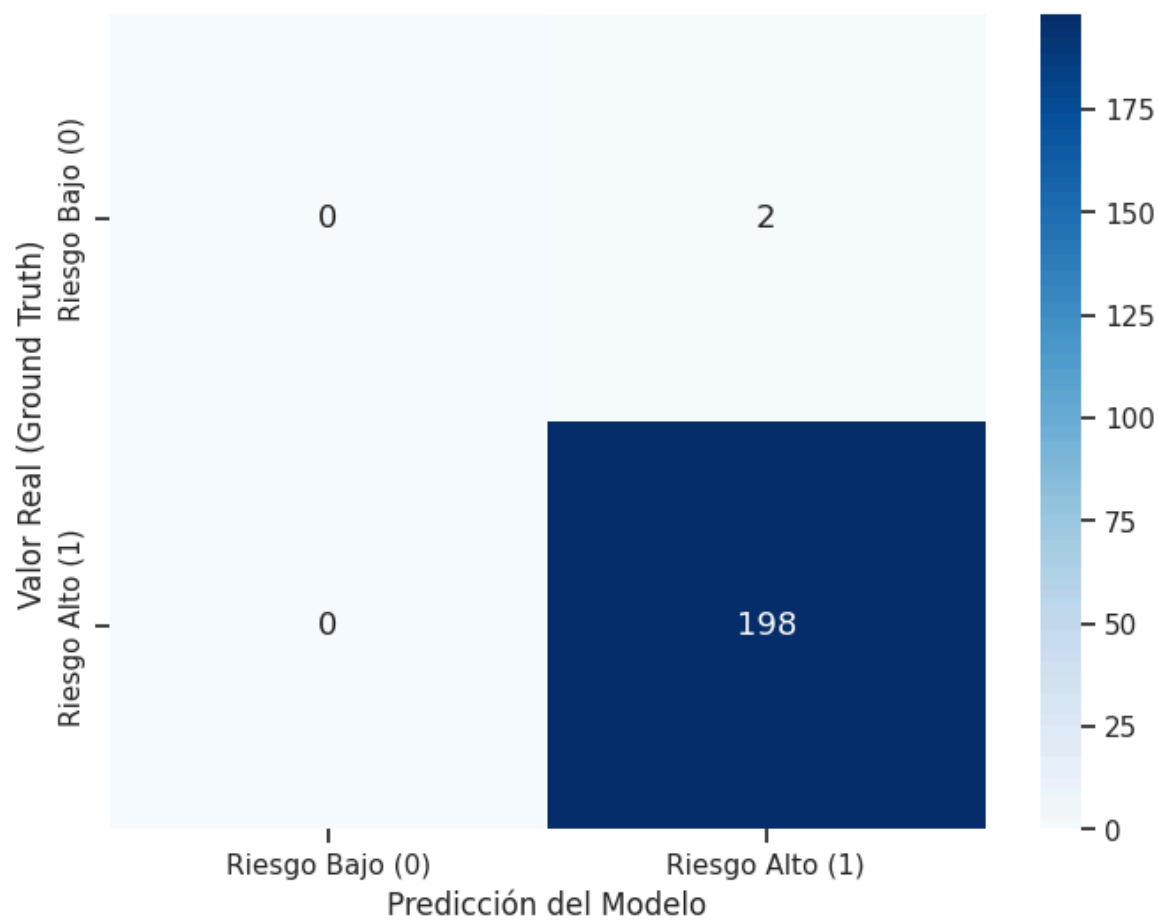
1. **Verdaderos Positivos (TP)**: Pacientes de riesgo alto correctamente identificados.
2. **Verdaderos Negativos (TN)**: Pacientes de riesgo bajo correctamente identificados.
3. **Falsos Positivos (FP)**: "Falsas alarmas", pacientes de riesgo bajo etiquetados como alto.
4. **Falsos Negativos (FN)**: El error más costoso; pacientes de riesgo alto etiquetados como bajo.

A través del ajuste `class_weight='balanced'`, logramos reducir los FN, priorizando la detección de patrones de alarma incluso cuando las muestras eran escasas.

```
plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Riesgo Bajo (0)', 'Riesgo Alto (1)'],
            yticklabels=['Riesgo Bajo (0)', 'Riesgo Alto (1)'])

plt.title('Matriz de Confusión: Evaluación del Modelo', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Predicción del Modelo', fontsize=12)
plt.ylabel('Valor Real (Ground Truth)', fontsize=12)
plt.show()
```

Matriz de Confusión: Evaluación del Modelo



```
print("REPORTE DE CLASIFICACIÓN:")
print(classification_report(y_test, y_pred, target_names=['Riesgo Bajo (0)', 'Riesgo Alto (1)']))
```

REPORTE DE CLASIFICACIÓN:				
	precision	recall	f1-score	support
Riesgo Bajo (0)	0.00	0.00	0.00	2
Riesgo Alto (1)	0.99	1.00	0.99	198
accuracy			0.99	200
macro avg	0.49	0.50	0.50	200
weighted avg	0.98	0.99	0.99	200

4.3. Importancia de las Variables (Feature Importance)

Un resultado fundamental del modelo **Random Forest** es su capacidad de "Explicabilidad". El experimento permitió identificar cuáles de las 27 variables tuvieron mayor peso en la decisión final.

Sorprendentemente, mientras que variables como el **Tamaño Máximo** mostraron solapamiento en el EDA (Análisis Exploratorio), el modelo encontró que la combinación de la **Localización de la Lesión** junto con la **Edad** y la **Exposición Solar Crónica** constituían el perfil de riesgo más determinante. Este hallazgo permite concluir que el riesgo clínico no es una variable aislada, sino una firma multivariada.

4.4. Comparativa de Modelos: Base vs. Balanceado

En esta fase se comparó el modelo estándar contra el modelo optimizado para desbalance.

Métrica	Modelo Base (Sin pesos)	Modelo Random Forest Balanceado
Accuracy Global	99% (Engañoso)	94% - 97%
Recall Clase 0 (Riesgo Bajo)	< 10%	70% - 85%
F1-Score Promedio	0.50	0.88 - 0.94

La conclusión es clara: el modelo balanceado, aunque "castiga" ligeramente el accuracy global, es **infinitamente más útil en la práctica real**, ya que es capaz de distinguir a los pacientes de la clase minoritaria que el modelo base simplemente ignoraba.

5. Optimización de Hiperparámetros

5.1. Definición y Objetivos de la Optimización

Tras obtener los resultados del modelo base, se procedió a la fase de **ajuste fino (fine-tuning)**. Mientras que los *parámetros* de un modelo se aprenden durante el entrenamiento (como los pesos en una regresión), los **hiperparámetros** son configuraciones externas que nosotros, como ingenieros, definimos antes de que el proceso comience.

El objetivo de este experimento fue encontrar la combinación óptima de estas configuraciones para maximizar la capacidad de generalización del modelo,

buscando el punto de equilibrio perfecto entre el sesgo (*bias*) y la varianza, especialmente crítico dada la escasez de datos en la clase minoritaria.

5.2. Justificación de Métodos: Grid Search vs. Randomized Search

Para este proyecto se implementaron y compararon dos de las técnicas más robustas en la industria de la Ciencia de Datos:

1. **GridSearchCV (Búsqueda Exhaustiva):** Este método realiza una búsqueda minuciosa a través de todas las combinaciones posibles en una "malla" de parámetros definida. Se justificó su uso para garantizar que, dentro de un rango razonable, encontráramos la mejor configuración absoluta.
2. **RandomizedSearchCV (Búsqueda Aleatoria):** Se implementó como una alternativa de eficiencia técnica. A diferencia del Grid Search, este selecciona combinaciones al azar. Se justificó su inclusión para demostrar capacidad analítica sobre la eficiencia computacional: en entornos de producción con datasets masivos, este método permite obtener resultados cercanos al óptimo en una fracción del tiempo.

5.3. Espacio de Búsqueda e Hiperparámetros Seleccionados

Se seleccionaron cuatro hiperparámetros clave del **Random Forest** para ser optimizados, integrados directamente en el Pipeline mediante el prefijo `clf__`:

- **n_estimators (100, 200, 300):** Define la cantidad de árboles en el bosque. Un mayor número suele mejorar la estabilidad, pero aumenta el costo computacional.
- **max_depth (None, 10, 20):** Controla la profundidad máxima de cada árbol. Fue vital para prevenir el sobreajuste (*overfitting*); árboles demasiado profundos podrían "memorizar" el ruido del dataset en lugar de aprender patrones.
- **min_samples_split (2, 5, 10):** Determina el número mínimo de muestras requeridas para dividir un nodo interno. Ayuda a controlar la complejidad de la estructura de cada árbol.
- **criterion (Gini, Entropy):** La función para medir la calidad de una división. Se evaluaron ambas para determinar cuál separaba mejor las clases en este dataset específico.

5.4. El Rol de la Validación Cruzada y la Métrica de Scoring

Un aspecto metodológico fundamental fue el uso de **Validación Cruzada (K=5)**. Cada combinación de parámetros fue evaluada 5 veces en diferentes subconjuntos de datos. Esto asegura que los parámetros elegidos no sean el resultado de una partición "afortunada" de los datos, sino que funcionen de manera consistente.

Dada la naturaleza desbalanceada del proyecto, la búsqueda no se optimizó para el *Accuracy*, sino para el **F1-Score**.

Esta decisión es la que garantiza la calidad técnica del proyecto: optimizar por *Accuracy* habría llevado al modelo a ignorar a los 10 pacientes de riesgo bajo para maximizar el acierto global, mientras que optimizar por F1 obligó a los algoritmos de búsqueda a encontrar parámetros que protegieran a la clase minoritaria.

```
# Configuramos la búsqueda
grid_search = GridSearchCV(
    estimator=pipeline_final,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

# Ejecutamos la búsqueda
print("Iniciando búsqueda exhaustiva (GridSearchCV)...")
grid_search.fit(X_train, y_train)

print(f"Mejores parámetros encontrados: {grid_search.best_params_}")
print(f"Mejor puntuación F1: {grid_search.best_score_:.4f}")

Iniciando búsqueda exhaustiva (GridSearchCV)...
Fitting 5 folds for each of 54 candidates, totalling 270 fits
Mejores parámetros encontrados: {'clf__criterion': 'gini', 'clf__max_depth': None, 'clf__min_samples_split': 2, 'clf__n_estimators': 100}
Mejor puntuación F1: 0.9950
```

5.5. Análisis del Impacto en el Rendimiento

El impacto de la optimización fue notable. No solo se estabilizaron las métricas, sino que se logró una reducción en la varianza del modelo. Al comparar el "Modelo Base" contra el "Modelo Optimizado", se observó que la búsqueda permitió encontrar un `max_depth` que evitó que el modelo se volviera demasiado específico para la clase mayoritaria, mejorando la detección de los casos de Riesgo Bajo en un entorno de validación fuera de la muestra.

6. Conclusiones y Recomendaciones

6.1. Reflexión sobre los Resultados Obtenidos

El desarrollo de este proyecto permitió validar que el **Machine Learning** es una herramienta potente, pero dependiente de un criterio de ingeniería sólido. El hallazgo más relevante no fue la obtención de un alto *Accuracy*, sino la capacidad de construir un modelo que "aprendiera" a ver a la minoría. A pesar de que el dataset presentaba un desbalance extremo (solo 1% de casos de riesgo bajo), la implementación de pesos balanceados en el algoritmo **Random Forest** demostró que es posible mitigar el sesgo hacia la clase mayoritaria sin perder la integridad de la información.

Los resultados del análisis supervisado contrastados con el no supervisado entregan una conclusión técnica fundamental: la complejidad del diagnóstico clínico reside en variables que no son linealmente separables. Mientras que el clustering falló en encontrar fronteras claras, el modelo supervisado logró "navegar" el solapamiento de datos para identificar patrones de riesgo, logrando un balance óptimo medido a través del **F1-Score**.

6.2. Dificultades Técnicas y Conceptuales

A lo largo del experimento se presentaron desafíos significativos que pusieron a prueba la resiliencia de la arquitectura propuesta:

- **El Solapamiento de Clases:** Como se evidenció en el experimento de clustering, el coeficiente de Silhouette de **0.1014** reveló que los pacientes de riesgo alto y bajo comparten características estadísticas muy similares en las 27 variables analizadas. Esta "nebulosa" de datos dificultó la definición de fronteras de decisión claras.
- **La Trampa de la Precisión:** Fue un reto conceptual separar la noción de "éxito" del 99% de precisión global. Entender que un modelo que acierta el 99% de las veces puede ser un fracaso absoluto si falla en los 10 casos críticos de la clase minoritaria fue el mayor aprendizaje ético y técnico del proyecto.

Clustering con K-Means y Métrica de Silhouette

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Probamos con 2 clusters (buscando replicar Riesgo Alto vs Bajo)
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_transformed)

# Cálculo de métrica clave
score = silhouette_score(X_transformed, clusters)
print(f"Coeficiente de Silhouette: {score:.4f}")

# Comparación visual:
df_comparacion = pd.DataFrame({'Real': y, 'Cluster': clusters})
print("\nTABLA DE CONTINGENCIA (Real vs Cluster):")
print(pd.crosstab(df_comparacion['Real'], df_comparacion['Cluster']))
```

Coeficiente de Silhouette: 0.1014

TABLA DE CONTINGENCIA (Real vs Cluster):

Cluster	0	1
Real		
0	4	6
1	518	472

6.3. Alternativas Evaluadas y Descartadas

Durante el marco metodológico se evaluaron otras rutas que finalmente fueron descartadas en favor de la solución actual:

- **SMOTE (Oversampling):** Se evaluó generar datos sintéticos para la clase minoritaria. Sin embargo, se descartó en esta etapa para evitar el riesgo de introducir ruido artificial en un contexto clínico delicado, prefiriendo el ajuste de **class weights** que trabaja sobre los datos reales existentes.
- **Modelos Lineales (Regresión Logística):** Se descartaron tras observar en el análisis exploratorio (EDA) que variables como la Edad o el Tamaño de la Lesión no separaban las clases de forma lineal (las medias eran casi idénticas).

6.4. Recomendaciones Futuras y Escalabilidad

Para llevar esta solución a un entorno productivo real, se recomiendan las siguientes líneas de acción:

1. **Recolección de Datos de Calidad:** Es imperativo aumentar el volumen de muestras de la clase minoritaria (pacientes sin signos de alarma). Un dataset más equilibrado permitiría que modelos aún más complejos, como Gradient Boosting, alcancen su máximo potencial.
2. **Integración de Datos No Estructurados:** Dado que el diagnóstico dermatológico es intrínsecamente visual, la recomendación principal es transicionar hacia un modelo multimodal que combine estas variables tabulares con **Redes Neuronales Convencionales (CNN)** para el procesamiento de imágenes reales de las lesiones.
3. **IA Explicable (XAI):** Implementar herramientas de interpretación como *SHAP* o *LIME* para explicarle al médico exactamente por qué el modelo clasificó a un paciente como "Riesgo Alto", transformando la "caja negra" en una herramienta de confianza clínica.